Knowledge Efficient Federated Continual Learning for Industrial Edge Systems

Jiao Chen, Jiayi He, Jianhua Tang, *Senior Member, IEEE*, Weihua Li, *Senior Member, IEEE*, and Zihang Yin, *Member, IEEE*

***Abstract*—Recent advances in federated learning (FL) primarily focus on addressing inter-client data heterogeneity, implicitly assuming static data within each client. However, this assumption is inadequate for industrial edge systems (IES), which operate in dynamically changing environments and require real-time processing and analysis of voluminous time-series data generated by the Internet of Things. To bridge this gap, we propose MeCo, a novel federated continual learning (FCL) method for IES, designed to avoid forgetting past knowledge while continuously adapting to new task data. MeCo distinguishes itself from traditional FL by effectively addressing both inter-client and intra-client data heterogeneity through a knowledge-efficient strategy. Specifically, it includes: *Meta task-invariant knowledge consolidation,* which helps capture shared features across tasks to alleviate forgetting; *Consistent task-specific knowledge transfer,* which allows edge clients to extract relevant knowledge from a server-side knowledge pool, providing a jump-starting for the current task. Experimental results demonstrate that MeCo significantly outperforms other federated and/or continual learning approaches in real-world industrial fault diagnosis, achieving approximately 2% higher Mean Average Accuracy and being 1.74 times more cost-effective in server-to-client communication. These advantages, along with its robust performance in IES, indicate the potential of MeCo for facilitating edge-cloud collaborative learning in the future.**

***Index Terms*—Federated learning, fault diagnosis, continual learning, knowledge transfer, edge intelligence.**

I. Introduction

IN THE era of the Industrial Internet of Things (IIoT), a vast array of devices such as sensors, surveillance cameras, and automated machinery generate enormous amounts of data in industrial environments [1], [2], [3], [4]. This rapid data growth not only increases the complexity of data processing but also poses significant challenges to the traditional centralized training and data storage paradigms in deep learning [5], [6], while also exacerbating the need for data privacy protection. Federated learning (FL) [7] offers a privacy-preserving framework that allows disstributed clients (e.g., users or devices) to collaboratively learn shared models directly at the data source, thus avoiding the need to transmit large amounts of raw data to a central server.

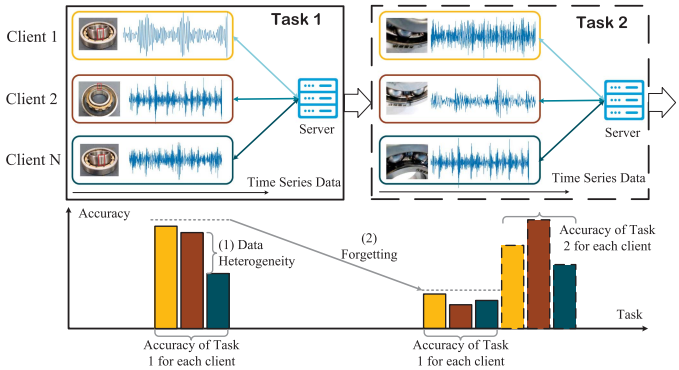


Fig. 1. In IES, there exists data heterogeneity both among clients and within clients. We found that: (1) heterogeneous local data leads to poor performance of the global model; (2) after completing the training on task 2, the model's performance on task 1 significantly decreases, resulting in catastrophic forgetting.

Although FL has made significant progress in addressing distributed machine learning and data privacy issues [8], its application in industrial edge systems (IES) still faces unique challenges. IES environments are characterized by significant dynamic changes, with data generated by devices accumulating in the form of continuous data streams. Existing FL research mainly focuses on addressing inter-client data heterogeneity, assuming that the intra-client data is static. However, this assumption is clearly not applicable in IES environments [9], [10].

In IES, in addition to inter-client data heterogeneity, the data within a client also changes over time, as shown in Fig. 1. If the model is retrained from scratch as new data emerges, the cost is very high, and the process is slow. Furthermore, there is the problem of catastrophic forgetting (CF), where the neural network, when learning new tasks, may overwrite weights associated with previous knowledge, leading to the loss of information acquired earlier. This phenomenon is particularly severe in industrial

Received 12 January 2024; revised 19 November 2024; accepted 16 February 2025. Date of publication 24 February 2025; date of current version 25 April 2025. The work of Jianhua Tang was supported in part by the National Key R&D Program of China under Grant 2024YFE0200500, in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2024A1515012615, and in part by the Department of Science and Technology of Guangdong Province under Grant 2021QN02X491. The work of Weihua Li was supported by the National Natural Science Foundation of China under Grant 52275111. Recommended for acceptance by Dr. Y. Zhang. *(Corresponding author: Jianhua Tang.)*

Jiao Chen and Jiayi He are with the Shien-Ming Wu School of Intelligent Engineering, South China University of Technology, Guangzhou 511442, China (e-mail: 202110190459@mail.scut.edu.cn; 202164020171@mail.scut.edu.cn).

Jianhua Tang is with the Shien-Ming Wu School of Intelligent Engineering, South China University of Technology, Guangzhou 511442, China, and also with Pazhou Lab, Guangzhou 510335, China (e-mail: jtang4@e.ntu.edu.sg).

Weihua Li is with the School of Mechanical and Automotive Engineering, South China University of Technology, Guangzhou 510641, China, and also with the Pazhou Lab, Guangzhou 510335, China (e-mail: whlee@scut.edu.cn).

Zihang Yin is with the China Academy of Information and Communications Technology, Beijing 100083, China (e-mail: yinzihang@caict.ac.cn).

Digital Object Identifier 10.1109/TNSE.2025.3544614

2327-4697 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.

applications requiring continuous monitoring and prediction, such as equipment fault diagnosis and maintenance.

Some existing federated continual learning (FCL) methods attempt to combine federated learning and continual learning to address these challenges. For example, some methods combine FedAvg [7] with replay-based continual learning methods, which rely on storing [11], [12] or generating [13] samples of previous classes and replaying these samples when training new data. However, the performance of these methods largely depends on the size of the sample set, and the continually growing number of samples may exceed the memory capacity of edge devices. Additionally, simply integrating continual learning methods into federated settings demonstrates poor performance, as they struggle to balance retaining previous knowledge and adapting to new knowledge while handling dynamic environments and real-time processing requirements.

Based on the limitations of existing methods, we propose that an effective FCL method should balance the following aspects: (1) Retaining previous knowledge (Stability): The model should effectively retain and utilize previously learned knowledge to avoid catastrophic forgetting. (2) Adapting to new knowledge (Plasticity): The model should be flexible enough to adapt to and learn from new task data.

To this end, we introduce MeCo, a novel FCL method designed to efficiently address both inter-client and intra-client data heterogeneity through a knowledge-efficient strategy. Specifically, MeCo consists of two key components:

Meta task-invariant knowledge consolidation: Helps capture shared features across tasks to mitigate forgetting.

Consistent task-specific knowledge transfer: Allows edge clients to extract relevant knowledge from a server-side knowledge pool, providing a *Jump-starting* mechanism for the current task. The concept of Jump-starting is borrowed from reinforcement learning [14]:

> *Jump-starting: learning an effective prior on the solution of current tasks (or accelerating future learning) by integrating knowledge from previous tasks with a small amount of new information.*

Through these two key components, MeCo can balance retaining previous knowledge and adapting to new knowledge while handling the dynamic changes and real-time processing requirements of IES environments.

A. Goal and Contributions

We study the problem of distributed learning from large-scale non-stationary task streams, and the pivotal challenge lies in how to learn from each edge client's private task streams quickly while alleviating forgetting. Our main contributions are:

- We propose MeCo, a novel FCL approach designed for IES. It incorporates **Me**ta task-invariant knowledge consolidation and **Co**nsistent task-specific knowledge transfer. This approach not only mitigates CF but also significantly enhances the model's capability for forward transfer.
- We introduce an innovative knowledge pool located on the server side, coupled with designed methods for knowledge selection and fusion. This approach is orthogonal to current rehearsal-based methods and is particularly suitable for resource-constrained IES, thereby improving their transferability.
- Our approach is extensively validated through a wide range of experiments and ablation studies, demonstrating its effectiveness and superiority in industrial fault diagnosis applications.

II. Related Works

A. Heterogeneity in Federated Learning

FL is a fast-evolving topic in the machine learning community, with applications spanning diverse fields, including industrial fault diagnosis [9], [15], traffic analysis [16], and smart agriculture [17]. Heterogeneity in FL can be categorized into model heterogeneity and data heterogeneity. HtFL enables collaborative learning among clients with different model architectures while preserving privacy [18].

Meanwhile, data heterogeneity drives significant research into personalization strategies within FL to address the diverse data distributions across clients. The single-model methods (fine-tune locally) learn a global model by designing fast convergence strategies, e.g., knowledge distillation [19], decentralized aggregation [8], [20] and analytic learning [21]. The multi-model methods aim to train personalized models for each edge client, e.g., split learning [22] and personalization layers [23].

A significant yet under-researched area within FL is the challenge posed by data heterogeneity that changes over time, commonly known as *data drift* [24]. Data drift can be fundamentally categorized based on its frequency. *One-data-drift:* Canonaco et al. [25] propose Adaptive-FedAvg, the server dynamically adjusts the learning rate of each client to account for data drift. This approach aims to maintain stable performance by promptly responding to changes in the data distribution. *Multi-data-drift:* Dupuy et al. [26] investigate FL in continuous environments, where devices generate data over time. They employ episodic replay to retain past information and mitigate CF caused by data distribution shifts.

B. Continual Learning for Edge

Continual learning (CL) draws inspiration from the human learning process. Just as humans leverage prior knowledge to grasp new concepts without forgetting the old ones, CL aims to emulate a similar pattern in machine learning models. Three primary perspectives address the challenge of CF in CL: (i) Regularization-based methods involve adding a regularization term to the loss function during training, which encourages the model to retain vital information learned from previous tasks while adapting to new tasks [27], [28], [29]. (ii) Replay-based methods store specific samples or representations from past tasks. During the learning of new tasks, these stored samples are replayed to help with retention [12]; (iii) Parameter isolation approaches [30] that dynamically adapt the model structure to the new task, ensuring the performance of previous tasks remains intact.

Edge computing places higher expectations on the efficiency, sustainability, and adaptability of learning algorithms in edge

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.

devices under changing data distributions [11], [24]. For instance, Zhuang et al. [31] propose an analytic exemplar-free continual learning method that efficiently adapts to new data without storing past samples. Similarly, Zhou et al. [32] showed that pre-trained models can be adapted to incremental learning with minimal resource consumption, enhancing sustainability and adaptability. These approaches underscore the critical requirements for edge-based learning algorithms.

C. Federated Continual Learning

Currently, FCL methods primarily include approaches based on regularization and rehearsal. Moreover, FCL methods can be further categorized into synchronous and asynchronous FCL based on the communication pattern [33]. Regularization methods [34] mitigate forgetting by adding regularization terms to the loss function, constraining parameter optimization directions to preserve previous knowledge. However, their effectiveness is limited when handling highly heterogeneous data. Rehearsal methods [11], [35] alleviate forgetting by replaying exemplars of previous tasks, allowing the model to recall old knowledge while training new tasks. Despite their efficacy in preserving memory, they face significant challenges in data privacy and communication overhead.

In contrast, parameter decomposition methods divide model parameters into task-specific and globally shared parts, achieving both data privacy protection and efficient knowledge sharing. Especially in IES, these methods manage task-specific knowledge effectively, reducing task interference and enhancing model stability and generalization. Additionally, by transmitting only necessary task-specific parameters, they significantly reduce communication burdens, crucial for real-time processing in IES. Notably, MeCo is a parameter decomposition method, which integrates spatiotemporal knowledge to address data heterogeneity, preventing forgetting while continuously adapting to new tasks, making it particularly advantageous in IES.

Notations: In this paper, calligraphic letters are used to denote sets, and vectors and matrices are represented by lowercase and uppercase bold letters, respectively. The expectation is denoted by $\mathbb{E}(\cdot)$, and the Euclidean norm is represented by $\|\cdot\|$. The gradient is shown with $\nabla(\cdot)$, and the d-dimensional real spaces are denoted by $\mathbb{R}^d$. When discussing parameter decoupling, the symbol $\circ$ represents the composition of models or functions. Lastly, the size of vector $\boldsymbol{q}$ is expressed as $|\boldsymbol{q}|$. For convenience, we summarize the symbols in Table I.

TABLE I
MAIN NOTATIONS

$\mathcal{T}_i^{(t)}$	the training task for edge client i at the t-th round
$\mathbf{K}$	the set of all keys in the knowledge pool
$\mathbf{k}_i^{(t)}$	the extracted task embedding for task $\mathcal{T}_i^{(t)}$
$\mathcal{U}$	the space of feasible set of task-invariant parameters
$\mathcal{V}$	the space of feasible set of task-specific parameters
$\mathbf{q}_i$	the parameters of edge client i
$\boldsymbol{u}_i^t$	the task-invariant parameters of client i at the t-th round
$\boldsymbol{v}_i^t$	the task-specific parameters of client i at the t-th round
$\mathbf{B}$	the knowledge pool containing all key-value pairs
$\Pi(\cdot)$	the similarity function to measure task features
$\mathbf{B}_i^{(t)}$	the consistent parameters selected for task t of client i
$\alpha_i^{(t)}$	attention on $\mathbf{B}_i^{(t)}$ to take a weighted combination
$A_c^{(t)}$	the average accuracy of edge client i at the t-th round
$F_c^{(t)}$	the forgetting of edge client i at the t-th round

III. Problem Formulation

In this section, we first provide a brief background on FL and CL. We then discuss the mathematical formulation of our proposed FCL, laying the foundational constructs for our discourse. Building upon this, we introduce an approach to enhance FCL, leveraging task-invariant parameters alongside task-specific parameters to refine the learning process.

A. Problem Definition

FL Formulation: The common FL with a central server and N edge clients (e.g., users or edge devices) can be captured via the following formulation:

$$\min_{(\mathbf{q}_{1:N})\in\mathcal{Q}} \frac{1}{N}\sum_{i=1}^{N} F_i(\mathbf{q}_i;\mathcal{D}_i), \tag{1}$$

where F_i denotes the loss function and $\mathbf{q}_i$ represents the parameters associated with the i-th edge client. The space of feasible sets is denoted as $\mathcal{Q}$. The goal of FL is to train a shared model by minimizing the average loss across clients while preserving data privacy by keeping local data decentralized. The learning model $\mathbf{q}_i : \mathbb{R}^d \to \mathcal{Y}$ maps input sample $\boldsymbol{x}_i \in \mathbb{R}^d$ to their corresponding predicted label $\mathbf{q}_i(\boldsymbol{x}_i) \in \mathcal{Y}$, which we would like to resemble the ground truth y_i. The dataset $\mathcal{D}_i$ represents a collection of data points and their corresponding labels for the i-th edge client, where each data point consists of an input-output pair drawn from the local distribution. The error F_i captures the *expected risk* over the data distribution of the specific edge client i, i.e.,

$$F_i(\mathbf{q}_i;\mathcal{D}_i) := \mathbb{E}_{(\boldsymbol{x}_i,y_i)\sim\mathcal{D}_i}[\ell(\mathbf{q}_i(\boldsymbol{x}_i),y_i)], \tag{2}$$

where p_i is the data distribution of local edge client i. Here, the function $\ell : \mathcal{Y}\times\mathcal{Y}\to\mathbb{R}$ computes the discrepancy between $\mathbf{q}_i(\boldsymbol{x}_i)$ and y_i.

CL Formulation: CL refers to training machine learning models on non-stationary data distributions, where different tasks are presented sequentially. We consider each edge client i learns from its private arriving task stream $\{\mathcal{T}_i^{(1)},\cdots,\mathcal{T}_i^{(t)}\}$, where $\mathcal{T}_i^{(t)} = \{(\boldsymbol{x}_{i,j}^{(t)}, y_{i,j})\}_{j=1}^{|\mathcal{T}_i^{(t)}|}$ is a labeled dataset of the t-th round.[1] It is important to note that $\mathcal{T}_i^{(t)}$ is only available for edge client i on the t-th round, and the previous data are no longer accessible due to the limited storage space of the edge clients.

CL can be categorized into three classes based on the task transition scenario: task-CL, class-CL, and domain-CL [32]. Task-CL assumes known task identity during training and testing, while class-CL deals with continually arriving new classes. In contrast, domain-CL maintains the same set of classes for each task and only changes input data distribution. Since domain-CL is more common in industrial applications [36], our paper focuses on this setting.

[1] In this context, t is initially defined as a index for the task stream. Subsequently, in specific instances, t refers to the round of operations performed at that particular index.

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.

In the context of domain-CL, for an edge client i, given $\mathcal{T}_i^{(t)}$ and the up-to-date knowledge $\boldsymbol{q}_i^{(t-1)}$, the objective function of edge client i not only assesses performance on the current task but also alleviates forgetting of previous tasks. This is typically achieved by introducing a regularization term into the loss function, which penalizes deviations of the model parameters from previous tasks' parameters. Thus, the objective function f_i can be defined as:

$$f_i(\boldsymbol{q}_i^{(t)}; \boldsymbol{q}_i^{(t-1)}, \mathcal{T}_i^{(t)}) := F_i(\boldsymbol{q}_i^{(t)}; \mathcal{T}_i^{(t)}) + \lambda \mathcal{R}(\boldsymbol{q}_i^{(t)}, \boldsymbol{q}_i^{(t-1)}), \tag{3}$$

where $F_i(\boldsymbol{q}_i^{(t)}; \mathcal{T}_i^{(t)})$ assesses the model's performance on task $\mathcal{T}_i^{(t)}$, $\mathcal{R}(\boldsymbol{q}_i^{(t)}, \boldsymbol{q}_i^{(t-1)})$ is a regularization term, measuring changes in the model parameters $\boldsymbol{q}_i^{(t)}$ relative to the previous task's parameters $\boldsymbol{q}_i^{(t-1)}$ and λ is a hyperparameter that controls the impact of the regularization term.

The CL objective function needs to cater to the entire sequence of tasks, so the overall optimization target is the cumulative of f_i across all tasks, i.e.,

$$\min_{\boldsymbol{q}_i^{(t)} \in \mathcal{Q}} \sum_{t=1}^{N_i} f_i(\boldsymbol{q}_i^{(t)}; \boldsymbol{q}_i^{(t-1)}, \mathcal{T}_i^{(t)}), \tag{4}$$

where N_i is the total number of tasks learned by the edge client i. This form of objective function allows the model to be optimized across the entire task sequence, while considering the requirements to alleviate forgetting.

FCL Formulation: FCL aims to optimize the learning process of each edge client from their private task streams, focusing on rapid knowledge acquisition while effectively minimizing CF. The primary goal of FCL is to learn a global model that integrates knowledge from multiple clients and performs well on incremental tasks. To achieve this, the central server provides two crucial services: (i) Aggregate the parameters sent by edge clients and broadcast the results to the edge clients, and (ii) store the privacy-agnostic knowledge of historical tasks. In IES, the central server typically has ample storage capacity to store all task knowledge. However, the computational limitations of edge clients prevent them from revisiting all knowledge during training, particularly when learning new tasks. The optimization objective of FCL is formulated as:

$$\min_{\boldsymbol{q}_{1:n}^{(t)} \in \mathcal{Q}} \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{N_i} f_i(\boldsymbol{q}_i^{(t)}; \boldsymbol{q}_{1:n}^{(t-1)}, \mathcal{T}_i^{(t)}). \tag{5}$$

The primary distinction between (4) and (5) lies in their knowledge utilization. While (4) relies solely on the knowledge of a single edge client, (5) leverages the comprehensive spatiotemporal knowledge available within the federated system.

B. Parameter Decomposition

Existing FCL methods primarily focus on reducing interference [37] and alleviating CF [35]. However, the learning efficiency holds equal importance in FCL. This efficiency crucially depends on the approach to knowledge transfer. Traditional knowledge transfer in CL relies on full model transfer, which involves periodically rehearsing all model weights [34]. However, this approach poses computational and communication burdens (as the parameter complexity is linear $\mathcal{O}(N \times |\mathbf{q}|)$) on edge devices and is prone to overfitting when dealing with numerous clients who have similar data distributions. To tackle these challenges, we adopt a parameter decoupling strategy that divides the model's parameters into task-invariant and task-specific parameters.

Specifically, task-invariant parameters denoted as $\boldsymbol{q}_{\boldsymbol{u}} : \mathbb{R}^d \to \mathbb{R}^b$, are designed to capture the shared knowledge across different tasks by mapping data points to a lower-dimensional space. On the other hand, task-specific parameters, denoted as $\boldsymbol{q}_{\boldsymbol{v}} : \mathbb{R}^b \to \mathcal{Y}$, concentrate on the unique aspects of each task, mapping from the lower-dimensional representation space to the output label space. For the sake of brevity and clarity in the subsequent discussions, we will refer to the task-invariant parameters $\boldsymbol{q}_{\boldsymbol{u}}$ simply as $\boldsymbol{u}$, and the task-specific parameters $\boldsymbol{q}_{\boldsymbol{v}}$ as $\boldsymbol{v}$. This notation simplifies the mathematical expressions while preserving the distinction between the two types of knowledge representation within the model.

The model for each edge client i in the t-th round, denoted as $\boldsymbol{q}_i^{(t)}$, is the composition of the task-invariant parameters $\boldsymbol{u}_i^{(t)}$ and the task-specific parameters $\boldsymbol{v}_i^{(t)}$. With these definitions, we can rewrite the optimization objective of FCL in (5) as

$$\min \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{N_i} f_i \left((\boldsymbol{u}_i^{(t)} \circ \boldsymbol{v}_i^{(t)}); (\boldsymbol{u}_{1:n}^{(t-1)} \circ \boldsymbol{v}_{1:n}^{(t-1)}), \mathcal{T}_i^{(t)} \right), \tag{6}$$

where $\boldsymbol{u}_{1:n}^{(t)} \in \mathcal{U}$ and $\boldsymbol{v}_{1:n}^{(t)} \in \mathcal{V}$ donates the space of feasible set of task-invariant and task-specific parameters. In this paper, we use various pre-trained models, e.g., ResNet [38] and ViT [39]. Given the limited computational power of the edge clients, we adopt the last few layers of the pre-trained models as task-specific parameters and the rest as task-invariant parameters.

A significant benefit of this parameter decoupling is its flexibility: we can store and reuse task-specific parameters, which facilitates addressing the optimization problem (3) significantly. In response, our focus shifts towards investigating a scalable method named MeCo, detailed in Section IV.

IV. Proposed Method

We now illustrate the learning procedure for problem (6). The server and clients aim to learn the task-invariant parameters iteratively, while the i-th client aims to learn its task-specific parameters locally. Our proposed approach, MeCo, alternates between the clients' update and server update on each communication round. The overview of MeCo is shown in Fig. 2.

A. Meta Task-Invariant Knowledge Consolidation

In neural networks, sequential learning of multiple tasks often leads to the challenge of CF, where acquiring new knowledge results in the loss of previously learned information. This phenomenon severely affects the model's performance in handling previously encountered tasks. Direct training of task-invariant parameters $\boldsymbol{u}$ end-to-end is inefficient due to the requirement

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.

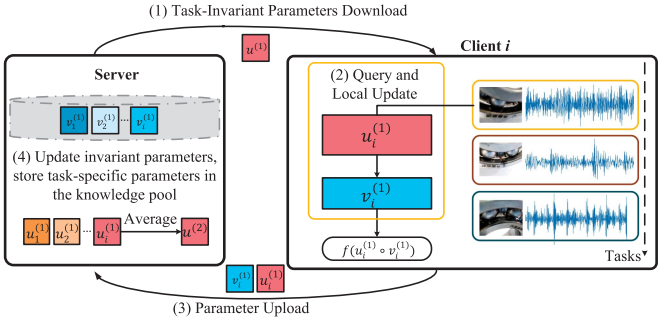


Fig. 2. Schematic diagram of the proposed MeCo. The client utilizes the knowledge pool of the central server to update task-specific parameters and then uploads the updated parameters back to the central server. This process comprises both inner and outer update and is carried out iteratively. We illustrate the Query process in Fig. 3.

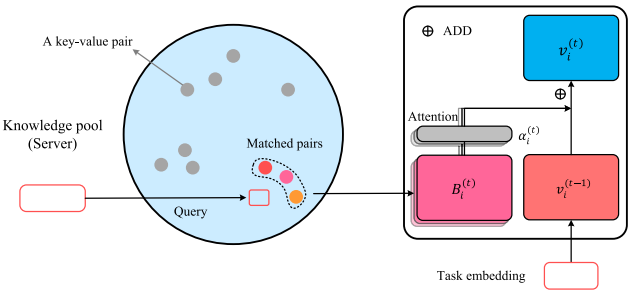


Fig. 3. Illustration of knowledge pool and consistent task-specific knowledge transfer. The server selects a subset relevant to the current task from the key-value pairing knowledge pool based on our proposed mechanism according to (8). Then, the edge client appends the selected task-specific parameters to the local $\boldsymbol{v}$ according to (9) to build a jump-starting for new tasks.

of extensive samples and multiple training epochs to achieve satisfactory convergence while also struggling to develop robust representations alleviate forgetting.

To effectively tackle these challenges, particularly the efficient training of task-invariant parameters $\boldsymbol{u}$, we turn to initialization-based meta-learning methods such as MAML [40]. These approaches are designed to enhance the neural network's fine-tuning capability, allowing for more efficient adaptation to a sequence of tasks. By focusing on the initialization of $\boldsymbol{u}$, these methods facilitate a learning process that accelerates convergence and significantly mitigates the impact of CF, thereby improving the model's ability to retain and utilize knowledge across varying tasks.

Recent advances in CL, which have incorporated meta-learning principles, provide further insights into enhancing the training of task-invariant parameters. Methods that align the current gradient with previous tasks' gradients [41] or focus on encoder learning to retain earlier tasks' knowledge [42] offer promising strategies. These approaches help tackle the forgetting problem and form a more stable foundation for task-invariant parameters, thus facilitating their role in capturing shared knowledge across different tasks.

We employ a two-stage approach to effectively learn the task-invariant parameters $\boldsymbol{u}$: inner and outer updates. These updates refine the parameters to better adapt to a sequence of tasks while alleviating forgetting.

Inner Update: Prior to training, each edge client i receives global task-invariant parameters $\boldsymbol{u}^{(t)}$ from the server and sets $\boldsymbol{u}_i^{(t)} = \boldsymbol{u}^{(t)}$. Subsequently, the client randomly splits the task $\mathcal{T}_i^{(t)}$ into support subset $\mathcal{T}_{i,s}^{(t)}$ and query subset $\mathcal{T}_{i,q}^{(t)}$. Diverging from the typical *N-way K-shot* setting in MAML, our method allows edge clients to benefit from learning with a limited number of examples, employing $\boldsymbol{u}_i^{(t)}$ to produce a b-dimensional embedding $\mathbf{k}_i^{(t)}$. This embedding is later sent to the central server to participate in task-specific knowledge queries, enhancing the model's ability to generalize across different tasks.

To ensure clarity and continuity, we will detail this process and its implications for the model's generalization capability in the subsequent sections. The discussions will focus on how the limited number of examples used in our setup provides sufficient information for the embedding generation and how the querying process with the central server contributes to refining the task-specific parameters $\boldsymbol{v}_i^{(t)}$.

Outer Update: Edge client i updates the task-invariant parameters $\boldsymbol{u}^{(t)}$ with the help of stable task-specific parameters. These meta-parameters $\boldsymbol{u}$ are refined during the meta-training phase and then fixed during the meta-testing phase, ensuring adaptability to new tasks while retaining knowledge of past tasks.

B. Consistent Task-specific Knowledge Transfer

In this section, we introduce consistent knowledge transfer. This strategy employs task embedding to efficiently query the server-side knowledge pool (the left side of Fig. 3) for relevant information. It selects consistent knowledge for new tasks (the right side of Fig. 3). We leverage the correlation between tasks to curate task-specific parameters, facilitating effective jump-starting for new tasks.

Knowledge Pool: The knowledge pool, illustrated in Fig. 3, is a pivotal component of our approach, designed to expedite forward transfer in task learning and to address the forgetting issue to some extent. It aims to facilitate knowledge sharing among similar tasks, accelerating the learning process for new tasks. By storing task-specific parameters, the knowledge pool enables a flexible composition of inputs for the model, providing seamless access to task-specific knowledge during training and inference phases and enhancing the model's learning capabilities and adaptability.

Assuming the server maintains a knowledge pool consisting of M pieces of knowledge, which encapsulates the knowledge of all historical tasks across all edge clients as key-value pairs, denoted by $\mathbf{B} = \{(\mathbf{k}_1, \boldsymbol{v}_1), (\mathbf{k}_2, \boldsymbol{v}_2), \ldots, (\mathbf{k}_M, \boldsymbol{v}_M)\}$. Here, each key $\mathbf{k}_i \in \mathbb{R}^b$ is a b-dimensional vector that represents the embedded knowledge of tasks. Importantly, the corresponding value $\boldsymbol{v}_i$ is a task-specific parameter derived from historical tasks. It is crucial to note that once stored in the knowledge pool, these parameters are no longer directly associated with their original edge client or the specific learning round. This abstraction allows the knowledge pool to manage and utilize these parameters more

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.

generalized and efficiently, facilitating knowledge sharing and transfer across different tasks and contexts. The set of all keys in the knowledge pool is denoted as $\mathbf{K} = \{\mathbf{k}_i\}_{i=1}^{M}$. This structure of key-value pairs in the knowledge pool allows the server to swiftly retrieve knowledge relevant to the current task.

Task Embedding: Tasks or datasets can be represented as fixed dimensional vectors, providing insights into the nature of these tasks and their interrelations. Prior to training a task $\mathcal{T}_i^{(t)}$, each client leverages task-invariant parameters to process a subset $\mathcal{T}_{i,s}^{(t)}$, thereby generating a b-dimensional task embedding. This embedding encapsulates the task's distribution, calculated by averaging the feature representations of the subset's data points. Importantly, only a small number of samples are required to estimate this distribution accurately; for instance, in the CWRU dataset, only 32 samples are sufficient [43].

To elaborate, for each client i on the training task at round t, the task embedding $\mathbf{k}_i^{(t)}$ is obtained by applying the task-invariant extraction layer $\boldsymbol{u}_i^{(t)}$ to each data point $(\boldsymbol{x}_j^{(t)}, y_j^{(t)}) \in \mathcal{T}_{i,s}^{(t)}$ and averaging the outputs as follows:

$$\mathbf{k}_i^{(t)} = \frac{1}{|\mathcal{T}_{i,s}^{(t)}|} \sum_{j=1}^{|\mathcal{T}_{i,s}^{(t)}|} \{(\boldsymbol{u}_i^{(t)}(\boldsymbol{x}_j^{(t)}), y_j^{(t)})\}, \tag{7}$$

where $|\mathcal{T}_{i,s}^{(t)}|$ is the size of the task subset. $\mathbf{k}_i^{(t)}$ is then communicated to the central server.

In contrast to raw data, task embedding $\mathbf{k}_i^{(t)}$ offers a generalized and task-agnostic approach to capture task-relevant information. These task embeddings are more compact, making them easier to store, and have the advantage of preserving privacy by reducing the risk of sensitive information leakage during data transmission. Furthermore, by distilling the semantic content, task embedding can effectively reduce data transmission costs while retaining important information for task learning.

Knowledge Transfer: CL research suggests that tasks with moderate to low relational distance to the current task are of higher significance [44]. We make a mild assumption: the data distribution of a new task is likely to be encompassed within the aggregate data distribution of all historical tasks. This assumption facilitates the identification of pertinent knowledge for the current learning scenario. The central server mines knowledge from the knowledge pool relevant to the edge clients' tasks and employs a parameter-sharing policy to transfer this knowledge.

To assess the relevance of knowledge, we introduce a similarity function $\Pi : \mathbb{R}^b \times \mathbb{R}^b \to \mathbb{R}$, which utilizes cosine distance in this work. The server computes the similarity scores and selects the top-K most pertinent parameters to be sent to the edge client, as per the following equation:

$$\mathbf{K}_i^{(t)} = \underset{\{s_i\}_{i=1}^{K} \in [1,M]}{\arg\min} \sum_{i=1}^{K} \Pi(\mathbf{k}_i^{(t)}, k_{s_i}), \tag{8}$$

where $\mathbf{K}_i^{(t)}$ represents a subset of keys selected for task $\mathcal{T}_i^{(t)}$ from the knowledge pool, and K is a predefined hyperparameter. In scenarios where the task history is long, K can keep the computational complexity of the edge client model constant.

Algorithm 1: MeCo - Federated Continual Learning for Industrial Edge Systems.

Input: Task streams $\{\mathcal{T}_i^{(1:t)}\}_{i=1}^{N}$, Global share task-invariant parameters $\boldsymbol{u}^{(0)}$, Task-specific parameters $\{\boldsymbol{v}_i^{(0)}\}_{i=1}^{N}$, Knowledge pool $P \leftarrow \{\varnothing\}$

1 Initialize weights $\{\alpha_i^{(0)}\}_{i=1}^{N}$
2 **for** *round* $t = 1, 2, \cdots$ **do**
3 Server broadcasts $\boldsymbol{u}^{(t-1)}$ to all edge clients
4 **for** *each edge client* $i \in N$ *in parallel* **do**
5 Set $\boldsymbol{u}_i^{(t)} = \boldsymbol{u}^{(t)}$
6 $(\mathcal{T}_{i,s}^{(t)}, \mathcal{T}_{i,q}^{(t)}) \leftarrow \mathcal{T}_i^{(t)}$
7 Calculate $\mathbf{k}_i^{(t)}$ using Eq. (7)
8 Server calculates task similarity by Eq. (8)
9 Server selects top-K parameters, then distributes $\{B_i^{(t)}\}$ to each edge client
10 Set local task-specific parameters by Eq. (9)
11 $\boldsymbol{u}_i^{(t+1)}, \boldsymbol{v}_i^{(t)} = \text{ClientUpdate}\big(\boldsymbol{u}_i^{(t)}, \boldsymbol{v}_i^{(t)}\big)$
12 Edge client sends $\boldsymbol{u}_i^{(t)}, \boldsymbol{v}_i^{(t)}$ to the server
13 Server aggregates $\boldsymbol{u}^{(t)} = (1/N)\sum_{i=1}^{N} \boldsymbol{u}_i^{(t)}$
14 Store $(\boldsymbol{k}_i, \boldsymbol{v}_i^{(t)})_{i=1}^{N}$ in P

Attention weights $\alpha_i^{(t)}$ are assigned to use indirect knowledge from the knowledge pool to emphasize the task-specific knowledge corresponding to these selected parameters. The task-specific parameters $\boldsymbol{v}_i^{(t)}$ within the model of edge client i are composed of the follows: (i) $\boldsymbol{v}_i^{(t-1)}$, which encapsulates the knowledge acquired from previous local incremental tasks, (ii) $\mathbf{B}_i^{(t)} = \{\boldsymbol{v}_{s_i}\}, s_i \in \mathbf{K}_i^{(t)}$, which comprises the parameters with relevant knowledge from selectively chosen past tasks, and (iii) the attention mechanism's output $\alpha_i^{(t)}$, leading to the following update:

$$\boldsymbol{v}_i^{(t)} = \alpha_i^{(t)} \odot \mathbf{B}_i^{(t)} + \boldsymbol{v}_i^{(t-1)}. \tag{9}$$

Here, $\odot$ denotes element-wise multiplication, emphasizing the selective incorporation of relevant task-specific knowledge into the current learning task.

Remark 1: In scenarios where the task streams of all clients exhibit homogeneity, it is advantageous to train a single global model that can generalize effectively. However, existing methods like FedPer [23] overlook the integration of task-specific knowledge, leading to limited performance when dealing with scenarios with minimal heterogeneity. In contrast, our method selectively combines task-specific parameters, allowing it to perform well in scenarios where the distribution is both static and dynamic.

C. Learning Procedures

Algorithm 1 summarizes the steps of MeCo. Edge clients start by sampling mini-batches from new tasks $\mathcal{T}_{i,s}^{(t)}$ and $\mathcal{T}_{i,q}^{(t)}$ (line 6),

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: ClientUpdate$(\boldsymbol{u}, \boldsymbol{v}_i)$.

Input: Number of steps $\tau_{\boldsymbol{v}}, \tau_{\boldsymbol{u}}$, and step sizes $\gamma_{\boldsymbol{v}}, \gamma_{\boldsymbol{u}}$

1 Initialize $\boldsymbol{v}_{i,0} = \boldsymbol{v}_i$

2 **for** $k = 0, 1, \cdots, \tau_{\boldsymbol{v}} - 1$ **do**

3 $\quad \boldsymbol{v}_{t,k+1} = \boldsymbol{v}_{t,k} - \gamma_{\boldsymbol{v}} \widetilde{\nabla}_{\boldsymbol{v}} f_i\big((\boldsymbol{u} \circ \boldsymbol{v}_{t,k+1}); (\boldsymbol{u} \circ \boldsymbol{v}_{i,k}), \mathcal{T}^{(t)}_{(i,s)}\big)$

4 Update $\boldsymbol{v}_i^+ = \boldsymbol{v}_{i,\tau_{\boldsymbol{v}}}$ and initialize $\boldsymbol{u}_{i,0} = \boldsymbol{u}$

5 **for** $k = 0, 1, \cdots, \tau_{\boldsymbol{u}} - 1$ **do**

6 $\quad \boldsymbol{u}_{i,k+1} = \boldsymbol{u}_{i,k} - \gamma_{\boldsymbol{u}} \widetilde{\nabla}_{\boldsymbol{u}} f_i\big((\boldsymbol{u}_{i,k+1} \circ \boldsymbol{v}_i^+); (\boldsymbol{u}_{i,k} \circ \boldsymbol{v}_i^+), \mathcal{T}^{(t)}_{(i,q)}\big)$

7 Update $\boldsymbol{u}_i^+ = \boldsymbol{u}_{i,\tau_{\boldsymbol{u}}}$

8 **Return** $\big(\boldsymbol{u}_i^+, \boldsymbol{v}_i^+\big)$

and then input $\mathcal{T}^{(t)}_{i,s}$ into the task-invariant parameters to extract the task embedding (line 7).

The edge client then uploads the task embedding to the central server and requests the server to query for similar task-specific parameters. The central server computes the similarity between the tasks in the knowlege pool and the current tasks using a predefined similarity function (line 8). It then selects the top-K most relevant task-specific knowledge for each task (line 9) based on this similarity and sends it back to the edge clients.

The edge clients adaptively use this task-relevant knowledge and previously learned knowledge to optimize the model for new task (line 10). Once the training of the edge clients' models converges (line 11), the task-invariant parameters $\boldsymbol{u}$ are uploaded and aggregated on the server, while the task-specific parameters are uploaded and stored in the knowledge pool (lines 12-14).

Algorithm 2 outlines the update process of edge client. The stochastic gradients with respect to $\boldsymbol{u}$ and $\boldsymbol{v}$ are denoted as $\widetilde{\nabla}_{\boldsymbol{u}}$ and $\widetilde{\nabla}_{\boldsymbol{v}}$, respectively. The update process starts by fixing the task-invariant parameters $\boldsymbol{u}$ and updating the task-specific parameters $\boldsymbol{v}$ for a given number of steps τ_v (lines 2-3). After updating $\boldsymbol{v}$, the client fixes $\boldsymbol{v}$ and updates the task-invariant parameters $\boldsymbol{u}$ for τ_u steps (lines 5-6). This alternating update approach allows the model to learn task-specific features while preserving global consistency.

D. Communication and Computation Complexity

During the learning process, the client communicates with the server in two main steps. First, the client sends the task embedding to the server to request similar task-specific parameters. The server then responds to these requests by sending the top-K matching task-specific parameters back to the client, with the data volume for this process being approximately $\mathcal{O}(NKV)$. After receiving the updated task-specific parameters from the clients, the server updates the knowledge pool and aggregates the task-invariant parameter $\boldsymbol{u}$ before sending it back to the client. The data volume for this aggregation process is approximately $\mathcal{O}(NP + NKV)$.

The computational complexity of MeCo is derived from two main components: local training complexity and task-specific parameter update complexity. The local training complexity arises from the computational cost of training on the local data at each client. Assuming each client performs E epochs of training on n_c data points, with a total of P model parameters, the local training complexity is $\mathcal{O}(En_cP)$. The task-specific parameter update complexity is introduced when selecting the Top-K task-specific parameters. Assuming the size of each task-specific parameter is V, this part of the complexity is $\mathcal{O}(KV^2)$. Combining these two components, the overall computational complexity of MeCo is $\mathcal{O}(En_cP + KV^2)$.

TABLE II
COMPARISON OF COMMUNICATION AND COMPUTATION COMPLEXITY

Algorithm	Communication Complexity	Computation Complexity
FedAvg	$\mathcal{O}(NP)$	$\mathcal{O}(En_cP)$
FedAvg + EWC	$\mathcal{O}(NP)$	$\mathcal{O}(En_cP + P^2)$
FedAvg + MAS	$\mathcal{O}(NP)$	$\mathcal{O}(En_cP + P)$
FedAvg + iCaRL	$\mathcal{O}(NP)$	$\mathcal{O}(En_cP + n_c^2P)$
FedWeIT	$\mathcal{O}(NP + NTV)$	$\mathcal{O}(En_cP + P + P^2)$
MeCo	$\mathcal{O}(NP + NKV)$	$\mathcal{O}(En_cP + KV^2)$

Algorithmic Complexity: In Table II, we compare the algorithmic complexity of various algorithms, specifically on communication and computation complexity. In the table, N denotes the number of clients, P is the number of model parameters, E is the number of local training epochs, n_c is the number of data points per client, T is the number of tasks, U is the size of the task-invariant parameters, V is the size of the task-specific parameters per task, and K is the number of selected task-specific parameters in MeCo. Communication complexity refers to the amount of data exchanged between the server and clients per round. Computation complexity refers to the computational cost incurred by each client per round. It is observed that MeCo has reduced algorithmic complexity compared to "FedAvg + EWC" and FedWeIT.

V. EXPERIMENTS

This section focuses on the experimental settings and performance evaluation of our proposed method, MeCo, where we compare it with baseline methods. Additionally, we perform ablation studies to further validate the effectiveness of our approach. Although our method has been specifically validated in fault diagnosis applications, its versatility as a model-agnostic approach is worth noting. It enables its applicability across various industrial domains by utilizing different backbone networks.

A. Experimental Setting

Overlapped-FD dataset: Previous studies have mainly employed a single dataset for federated training across the clients, including CWRU and SCUT-FD. CWRU [45] is a widely used benchmark dataset for bearing fault diagnosis, and the data is collected using accelerometers from fan-side and drive-side deep groove ball bearings. SCUT-FD is a bearing fault diagnosis dataset for real industrial scenarios, details of which can be found in previous work [36]. To better approximate the real-world industrial environment, we validate our MeCo under task

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.

TABLE III
DATASET DETAIL OF OVERLAPPED-FD TASK, INCLUDING NUMBER OF SUB-TASKS AND CLASSES PER SUB-TASK

Overlapped-FD Dataset							
Dataset	**RPM**	**Num. Classed**	**Num. Tasks**	**Dataset**	**RPM**	**Num. Classes**	**Num. Tasks**
CWRU	1730	10	2	**SCUT-FD**	500	5	1
	1750	10	2		800	5	1
	1772	10	2		1100	5	1
	1797	10	2		1400	5	1
	1730+1750	10	2		500+800	5	1
	...	10	2		...	5	1
	1772+1797	10	2		1100+1400	5	1
Total		100	20	**Total**		50	10

sequences baseline, namely Overlapped-FD. Overlapped-FD consists of both CWRU and SCUT-FD datasets.

Specifically, the CWRU dataset includes three types of bearing failures, IF, OF, and RoF, as well as health conditions (NC). Four loads (0, 1, 2, 3) resulted in four different rotating speeds (1730, 1750, 1772, 1997 rpm). From a practical point of view, we used data at different speeds to simulate data drift. Furthermore, inspired by [46], we designed more complex diagnostic tasks by randomly combining data from two different rotating speeds (i.e., $C_4^2 = 10$ complex tasks). For the SCUT-FD dataset, we use the same division. The detailed descriptions of the task configuration are given in Table III. Eventually, Overlapped-FD contains 150 classes. After generating and processing the tasks (5 classes per task), we randomly assign them to multiple clients.

Similar to other experimental setting for CL [37], our experimental dataset does not consider task boundary information. However, task transition detection methods need to be introduced in realistic scenarios, as task transitions are not discrete.

Backbone Network: We primarily adopt ResNet18 [38] as the backbone network, without using any pre-trained weights. Considering the limited computational capabilities of edge clients, we adopt the last few layers of the network as task-specific parameters, while the remaining layers serve as task-invariant parameters. During the preprocessing of vibration signals collected by sensors, we first apply the continuous wavelet transform (CWT) to convert the time-domain signal into time-frequency representation [47]. As shown in Fig. 4, this conversion is performed at intervals of 1024 time steps. The frequency range of the vibration signal is 0-12 kHz. By using CWT for time-frequency representation, it provides high resolution, enabling the capture of detailed features of the signal. In addition, to verify the compatibility of our approach with different backbone networks, we also evaluated the performance of ResNet-50 and ViT [39].

Implementation Details: Training is conducted on two RTX-4090 GPUs (physical memory size of 128G). The hyperparameters used in our experiments include a batch size of 64, and criterion options with cross-entropy loss. For the ResNet model, the fine-tuned parts include the last residual block and the classifier. For the replay-based continual learning method iCaRL, the replay buffer size is set to 12,000. We utilize the Adam optimizer with a learning rate of 10^{-3} and a eight decay rate of 10^{-5} for training. Each communication round consists of 5 epochs, ensuring a comprehensive training experience. To

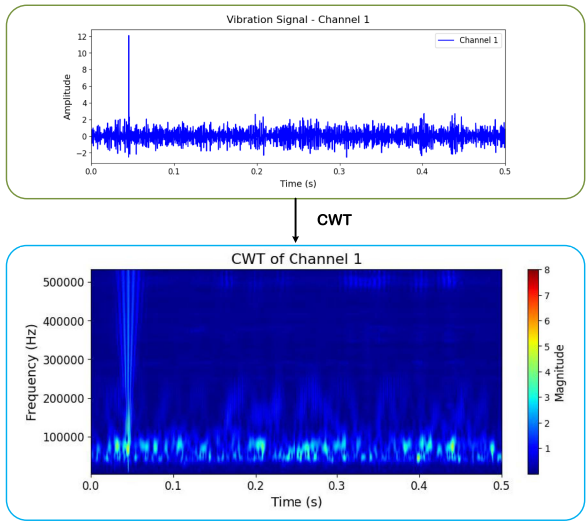


Fig. 4. Vibration signal and time-frequency representation.

prevent overfitting, we monitor the loss closely and terminate the training phase if there is no improvement for 3 consecutive epochs. In addition, we present the mean metrics aggregated over three iterations, with a 95% confidence interval.

Performance Metric: We adopt the following performance metrics to evaluate MeCo, which are in line with previous works [48].

1) *Macro-averaged Accuracy (MAA):* Accuracy measures the proportion of correctly classified instances, and it is a commonly used metric in machine learning. We measure the averaged accuracy of all the tasks after the completion of a CL at task t by $A_i^{(t)} = \frac{1}{t}\sum_{j=1}^{t} a^{t,j}$, where $a^{t,j}$ is the test accuracy of task j after learning t tasks. The MAA at task t is defined as the average of $A_i^{(t)}$ over all edge clients as follows

$$A^{(t)} = \frac{1}{N}\sum_{i=1}^{N} A_i^{(t)}, \tag{10}$$

where N is the number of clients.

2) $F1_{macro}$: The mean F1-score of participating clients, i.e., $F1_{macro} = 1/n\sum_{i=1}^{N} F1_i$, where $F1_i$ is the F1-score of client i.

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
AVERAGED PER-TASK PERFORMANCES ON OVERLAPPED-FD

Method	Type	MAA(%)	F1_macro(%)	Forgetting	S2C	C2S
STL	Baseline	82.36 (± 0.73)	79.52 (± 0.67)	0.11 (± 0.02)	NaN	NaN
EWC	Continual (Regularization)	80.82 (± 0.82)	77.86 (± 0.34)	0.08 (± 0.01)	NaN	NaN
MAS	Continual (Regularization)	80.63 (± 0.46)	77.75 (± 0.44)	0.07 (± 0.02)	NaN	NaN
iCaRL	Continual (Rehearsal)	82.42 (± 0.46)	78.55 (± 0.36)	0.05 (± 0.01)	NaN	NaN
OML	Continual (Meta-learning)	82.57 (± 1.75)	79.78 (± 1.57)	0.06 (± 0.03)	NaN	NaN
FedAvg	Federated	88.46 (± 1.68)	86.27 (± 1.95)	0.10 (± 0.01)	2.6GB	2.6GB
FedAvg + EWC	Federated Continual	81.16 (± 2.14)	78.29 (± 1.66)	0.09 (± 0.01)	2.6GB	2.6GB
FedAvg + iCaRL	Federated Continual	87.11 (± 2.55)	83.32 (± 1.35)	0.06 (± 0.01)	2.6GB	2.6GB
FedWeIT	Federated Continual	90.13 (± 0.59)	87.45 (± 0.46)	0.03 (± 0.01)	5.4GB	2.6GB
MeCo	**Federated Continual**	**92.26** (± 0.17)	**91.25** (± 0.34)	0.02 (± 0.01)	3.1GB	**2.6GB**

3) *Forgetting:* For task j, the maximum accuracy is achieved right after learning task j, which is $a^{j,j}$. As more tasks are learned, the accuracy might decrease. The forgetting rate for task j after learning t tasks, denoted as $F^{t,j}$, can be defined as the difference between the maximum accuracy of task j and its current accuracy: $F^{t,j} = a^{j,j} - a^{t,j}$. The overall forgetting rate for edge client i is defined as the average of the forgetting rate of all tasks learned up to the N_i-th task, which can be expressed as

$$F_i^{N_i} = \frac{1}{N_i - 1} \sum_{j=1}^{N_i - 1} F^{N_i,j}, \tag{11}$$

where N_i is the total number of tasks learned by the edge client i.

B. Performance Comparison

Baselines: We compare MeCo to the following baseline methods.

1) Single Task Learning (**STL**) refers to each task is associated with a distinct model. The models are trained and tested exclusively on individual tasks, without the consideration for knowledge transfer or retention across multiple tasks.
2) **EWC** [27] is a CL method that uses a regularization term to retain important parameters from previous tasks. It starts from a Bayesian perspective and uses the Fisher information matrix to approximate the importance of each parameter.
3) **MAS** [28] proposes memory perception synapses and measures the importance of parameters in terms of their effect on output outcomes.
4) **iCaRL** [12] constructs a data buffer to store a small number of exemplars per class to train with data from the current task.
5) **OML** [42] proposes a meta-learning objective to learn a robust representation under online updates to alleviate forgetting and facilitate future learning.
   The above benchmark for CL represents the execution of CL on a single client.
6) **FedAvg** [7] is the most commonly used method for FL, which learns a single global model for all clients.
7) **FedWeIT** [37] extends the concept of regularization-based CL into the FL setting, aiming to mitigate interference among edge clients. This approach introduces mask and attention mechanisms between local and global models, allowing for selective updates in the local model.

Results on Accuracy: Table IV compares the performance of the different methods. MeCo outperforms other baseline methods in terms of accuracy and alleviate forgetting. Compared to the most competitive baseline method, FedWeIT, our method achieves approximately 2% higher MAA while significantly reducing the communication costs for server-to-client (S2C) communication. Specifically, our method is 1.74 times more cost-effective regarding S2C communication compared to FedWeIT. These superior performance and cost advantages make MeCo particularly well-suited for IES. A detailed analysis of each performance metric is presented below.

The average accuracy of various edge clients under different CL and FCL methods is evaluated and presented in Fig. 5. Our method demonstrates higher average accuracy than the baseline approach throughout 60 training rounds. Moreover, Fig. 5 shows that MeCo consistently achieves a more rapid convergence compared to benchmark methods such as EWC, MAS, iCaRL, OML, FedAvg, and FedWeIT, particularly in the later stages of the communication rounds. This enhanced performance can be attributed to the efficient use of task-specific parameters in the MeCo knowledge pool, which provides the model with a beneficial jump-starting in adapting to new tasks. Furthermore, the average accuracy of EWC, FedAvg, and MeCo is evaluated across different communication rounds, as depicted in Fig. 6. By observing the forgetting rate of each task, we conclude that as training progresses with new tasks, the accuracy steadily improves with fewer fluctuations. This trend highlights the robustness of our method. Specifically, a lower forgetting rate indicates a more stable learning process, thereby validating the effectiveness of our approach in continual learning. The effectiveness of our approach in alleviating forgetting during FCL can be attributed to efficient knowledge sharing and accumulation, which helps preserve previously learned information.

The naive federated learning method (i.e., FedAvg) can more effectively learn from distributed data and improve model performance without interference from additional continual learning mechanisms. iCaRL requires storing and replaying old data samples to maintain previously learned knowledge, which may

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.

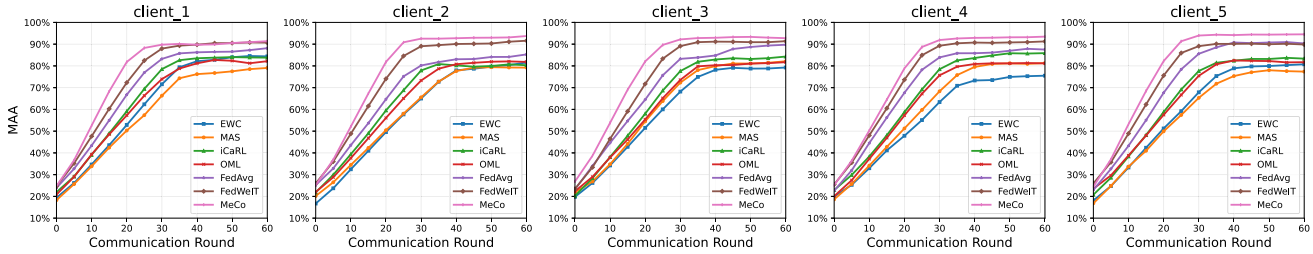


Fig. 5. The comparison of the averaged accuracy over the 6 tasks of edge clients during 60 communication rounds.

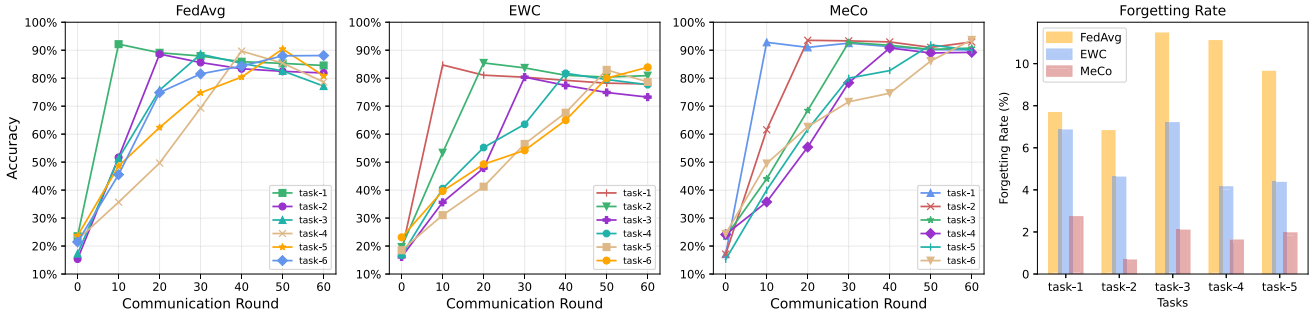


Fig. 6. The analysis of per-task accuracy across 60 communication rounds for edge client 5.

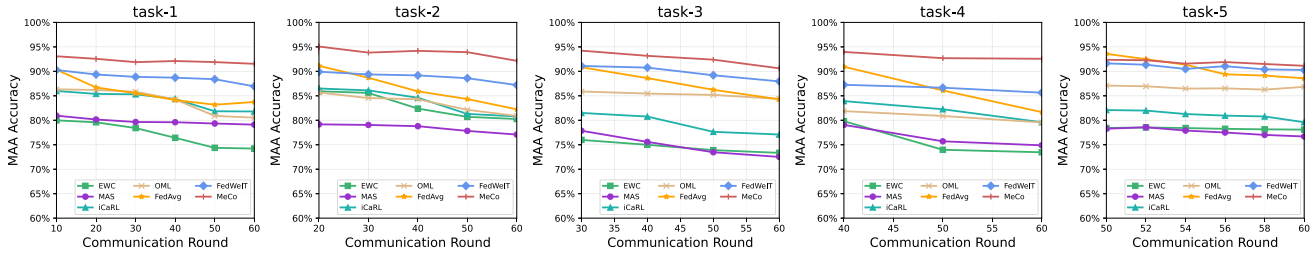


Fig. 7. The comparison of forgetting among trained tasks across multiple communication rounds.

lead to resource allocation conflicts in federated learning. However, FCL methods can effectively address these issues, thus achieving better performance.

Results on Knowledge Transfer: FL-based methods, such as FedAvg, FedWeIT, and our proposed MeCo, consistently demonstrate superior accuracy compared to locally trained methods like STL, EWC, MAS, and iCaRL. This finding underscores the effectiveness of knowledge exchange among edge clients in mitigating the limitations posed by limited local data. Moreover, our MeCo approach surpasses other FL-based methods like FedAvg and FedWeIT in accuracy. This notable performance can be primarily attributed to task-specific knowledge sharing among edge clients, effectively reducing interference caused by the transfer of unrelated knowledge between client models.

Results on Forgetting: Our proposed method, MeCo, effectively mitigates the issue of forgetting previously acquired knowledge during continuous training of subsequent tasks. This can be observed in Fig. 7, where the accuracy trend of the FedAvg approach shows a gradual decline, indicating forgetting of knowledge from past tasks. In contrast, FCL methods like FedWeIT and MeCo demonstrate the ability to retain knowledge from past tasks, leading to higher accuracy.

TABLE V
ABLATION ANALYSIS: THE IMPACT OF OUR CORE DESIGN ON PERFORMANCE

Variant	MAA(%)	F1_macro(%)
MeCo	92.26	91.25
w/o C-K transfer	81.66 (-10.50)	79.2 (-11.05)
w/o Meta	85.74 (-6.42)	84.4 (-6.85)

C. Ablation Study and Analysis

We perform an ablation study to assess the impact of key components in our method on performance. The results, as presented in Table V, indicate the influence of each component. A significant decrease in accuracy is observed in the "*w/o*

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
PERFORMANCES UNDER DIFFERENT BACKBONES

Backbone	Method	MAA (%)	F1_macro (%)	Training Time (ms)	Inference Time (ms)
MobileNet	FedAvg	84.55	82.30	75	45
	FedWeIT	86.71	83.45	90	55
	MeCo	88.92	87.80	85	50
ResNet18	FedAvg	88.46	86.27	125	75
	FedWeIT	90.13	87.45	150	100
	MeCo	92.26	91.25	142	87
ResNet50	FedAvg	88.72	86.51	300	175
	FedWeIT	91.99	87.95	375	225
	MeCo	94.03	94.14	338	200
ViT-B/16	FedAvg	89.52	87.31	750	375
	FedWeIT	92.61	90.25	875	500
	MeCo	95.41	94.32	812	437

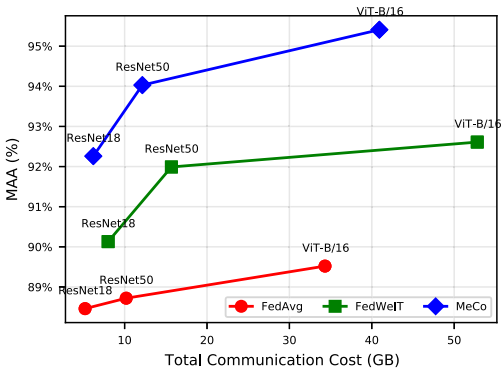


Fig. 8. Total communication cost and model accuracy.

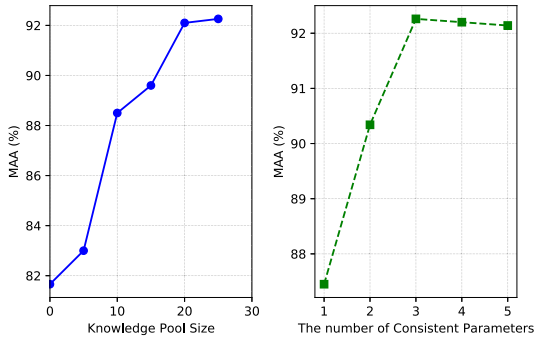


Fig. 9. Effect of knowledge pool size and the number of consistent parameters.

C-K transfer" setting, where Consistent task-specific knowledge transfer among edge clients is removed. Similarly, there is a notable decline in accuracy in the "*w/o* Meta" setting, where the Meta task-invariant knowledge consolidation is excluded. These findings highlight these components' crucial contributions to our method's performance.

Comparison of Communication Cost: As shown in Fig. 8, our MeCo approach achieves high accuracy with lower communication costs compared to traditional methods. Both FedAvg and FedWeIT show an increase in accuracy with higher communication costs, yet MeCo outperforms at a fraction of the cost. Notably, the complex models such as ViT and ResNet show that while higher complexity can yield better accuracy, this comes at the expense of increased communication overhead.

Performance with Different Backbones: We evaluate three network backbones, MobileNet (∼4.2 million parameters) [49], ResNet18 (∼11.7 million parameters), and ViT-B/16 (∼86 million parameters) [39], across three methods: FedAvg, FedWeIT, and our proposed MeCo. As shown in Table VI, MeCo consistently outperforms other baselines in accuracy while requiring lower communication costs than FedWeIT. The above results demonstrate the versatility and compatibility of MeCo with various network backbones.

In this experiment, we simulate 5 clients. The reported training and inference times represent the total duration to precess a single batch (batch size of 16) across all clients on the GPUs. The results indicate that, as the complexity of the backbone increases from ResNet18 to ResNet50 and ViT-B/16, both training and inference times significantly increase. For example, ResNet18 has a training time of only 125 milliseconds and an inference time of 75 milliseconds, making it well-suited for latency-sensitive applications. Based on the computational capabilities of edge clients, one can select appropriate backbone networks and leverage MeCo to achieve improved accuracy and reduced communication costs.

Effect of Knowledge Pool Size and Consistent Parameters: Fig. 9 shows the influence of knowledge pool size and the number of consistent parameters on the predictive accuracy for the Overlap-FD dataset. The x-axis of the left-plot delineates the range of knowledge pool sizes, extending from 5 to 30. An ascending trend is evident, signifying that augmenting the knowledge pool size correlates with an enhancement in the average accuracy. This correlation substantiates the notion that an enlarged knowledge pool can potentially facilitate superior model learning, thereby yielding more precise predictions.

The right-plot delineates the impact of varying quantities of consistent parameters, ranging from 1 to 5, on model accuracy. The depicted trend suggests an initial improvement in accuracy with an increase in the number of consistent parameters, followed by a decrease. This decrease likely occurs because incorporating too many consistent parameters introduces irrelevant knowledge, which may interfere with learning for new tasks. By selectively incorporating the most relevant external parameters,

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
PERFORMANCES UNDER DIFFERENT DISTANCE METRICS

Distance	MAA (%)	F1_macro (%)
Euclidean	91.13 (± 0.12)	91.05 (± 0.16)
KL	91.77 (± 0.09)	91.19 (± 0.10)
Cosine	92.26 (± 0.17)	91.25 (± 0.34)

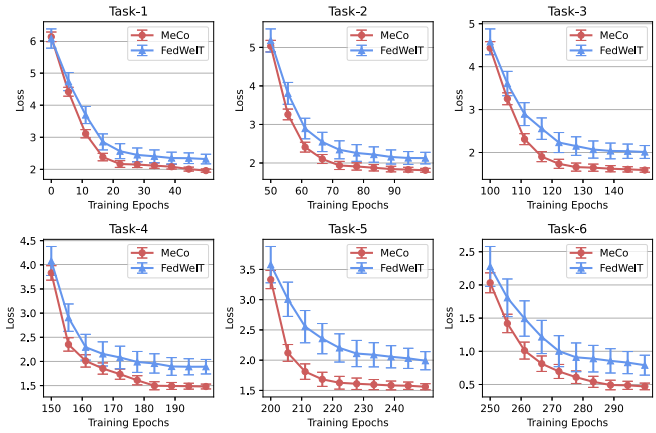


Fig. 10. Error loss across different tasks during training (client 5).

our approach facilitates forward transfer learning, thereby improving performance and ensuring effective knowledge transfer across tasks.

Performance under Different Distance Metrics: When constructing our knowledge pool for task-specific knowledge selection, we consider task characteristics and employ Cosine Distance to compute similarity. We also evaluate the retrieval accuracy of our method using Kullback-Leibler (KL) Divergence and Euclidean Distance. Results in Table VII show Cosine Distance outperforms other metrics, proving its effectiveness for task-relevant knowledge selection.

Training Loss: To further investigate the effectiveness of MeCo, we analyzed the error loss of different tasks during the training epoch. As illustrated in Fig. 10, we compared the loss from Task 1 to Task 6 during the training epoch between MeCo and FedWeIT. Our results show that MeCo achieves lower loss and converges faster than FedWeIT. This improvement is mainly because that MeCo enables edge models to continuously adapt to new tasks with minimal parameter changes while retaining previously learned knowledge.

Scalability Analysis: To validate the scalability of MeCo, we conducted experiments by increasing the number of clients and the number of tasks per client using the CIFAR-100 and Overlapped-FD datasets, respectively.

Increasing the number of clients typically introduces more diverse data, which helps enhance the generalization ability of the model. As shown in Table VIII, when the number of clients (N) increases from 5 to 10, the MAA of MeCo changes slightly but remains at a high level overall. For example, in the CIFAR-100 dataset, when N increases from 5 to 10, the MAA of MeCo decreases slightly from 70.01 to 69.95, and the forgetting rate increases from 2.75 to 3.40. This indicates that a moderate increase in the number of clients does not significantly affect performance. However, as the number of clients increases, the communication and computation overhead in the federated learning system also increases, which needs to be balanced.

TABLE VIII
PERFORMANCE EVALUATION WITH VARYING CLIENT AND TASK NUMBERS

Metric	Method	CIFAR-100		Overlapped-FD	
		N=5 N_i=10	N=10 N_i=10	N=5 N_i=6	N=5 N_i=12
MAA (%)	FedAvg	66.31	60.14	88.46	85.06
	FedAvg + iCaRL	67.98	63.19	87.11	85.42
	MeCo	70.01	69.95	92.26	91.72
Forgetting (%)	FedAvg	9.72	9.29	12.60	14.70
	FedAvg + iCaRL	3.91	4.40	9.46	9.70
	MeCo	2.75	3.40	2.20	2.93

The results of MeCo do not degrade with an increase in the number of tasks. For instance, in the Overlapped-FD dataset, when N_i increases from 6 to 12, the MAA of MeCo only slightly decreases from 92.26 to 91.72, and the forgetting rate increases from 2.20 to 2.93. This means that MeCo can still maintain performance even with the incremental addition of new data, i.e., without significant performance degradation due to the inclusion of new data.

VI. CONCLUSION

In this paper, we explored the challenges associated with federated continual learning in distributed settings, where participants face non-stationary task streams. We introduced MeCo, a novel learning framework built on the principles of generic knowledge accumulation and task-specific parameter transfer. A distinctive advantage of MeCo lies in its efficiency, by maintaining the knowledge pool in the cloud and allowing edge clients to perform queries only. Extensive experiments reveal that MeCo significantly reduces edge storage costs compared to replay-based methods, and achieves comparable, even better performance than prior arts on industrial fault diagnosis applications. Due to its simplicity and general applicability, MeCo holds great promise for enhancing edge-cloud collaborative learning in future scenarios. In this work, our evaluation primarily targets industrial fault diagnosis, the applicability of MeCo across other domains and tasks remains an area for further exploration. Therefore, future endeavors can also focus on extending our framework to accommodate a wider array of continual pre-training tasks.

REFERENCES

[1] L. Ren, J. Dong, S. Liu, L. Zhang, and L. Wang, "Embodied intelligence toward future smart manufacturing in the era of AI foundation model," *IEEE/ASME Trans. Mechatron.*, early access, Sep. 27, doi: 10.1109/TMECH.2024.3456250.
[2] X. Wu, J. Tang, and M. Siew, "Digital twin-assisted semi-federated learning framework for industrial edge intelligence," *China Commun.*, vol. 21, no. 5, pp. 314–329, May 2024.
[3] F. Chen, J. Tang, and Z. Yin, "Age of loop information with flexible transmission enabled communication and control co-design in industrial IoT," *China Commun.*, vol. 21, no. 11, pp. 40–55, Nov. 2024.

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.

[4] J. Tang, F. Chen, J. Li, and Z. Liu, "Learn to schedule: Data freshness-oriented intelligent scheduling in industrial IoT," *IEEE Trans. Cogn. Commun. Netw.*, vol. 11, no. 1, pp. 505–518, Feb. 2025.

[5] D. Liu, H. Kong, X. Luo, S. Huai, and W. Liu, "Edge intelligence: From deep learning's perspective," in *Proc. Digit. Manuf.*, Jan. 2024, pp. 169–211.

[6] Z. Zheng, Y. Li, H. Song, L. Wang, and F. Xia, "Towards edge-cloud collaborative machine learning: A quality-aware task partition framework," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, New York, NY, USA, Oct. 2022, pp. 3705–3714.

[7] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, Ft Lauderdale, FL, USA, Apr. 2017, vol. 54, pp. 1273–1282.

[8] M. Shen et al., "Secure decentralized aggregation to prevent membership privacy leakage in edge-based federated learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 3105–3119, May/Jun. 2024.

[9] J. Chen, J. Tang, and W. Li, "Industrial edge intelligence: Federated-meta learning framework for few-shot fault diagnosis," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3561–3573, Nov./Dec. 2023.

[10] J. Chen et al., "Towards general industrial intelligence: A survey on IIoT-enhanced continual large models," 2024, *arXiv:2409.01207*.

[11] L. Zhang, G. Gao, and H. Zhang, "Spatial-temporal federated learning for lifelong person re-identification on distributed edges," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 35, no. 2, pp. 1884–1896, Feb. 2025.

[12] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "iCaRL: Incremental classifier and representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Honolulu, HI, USA, Jul. 2017, pp. 2001–2010.

[13] J. He, J. Chen, Q. Liu, S. Dai, J. Tang, and D. Liu, "Continual learning with diffusion-based generative replay for industrial streaming data," in *Proc. IEEE/CIC Int. Conf. Commun. China*, Hangzhou, China, Aug. 2024, pp. 1015–1020.

[14] S. Li, H. Li, J. Zhang, Z. Wang, P. Liu, and C. Zhang, "IOB: Integrating optimization transfer and behavior transfer for multi-policy reuse," *Auton. Agents Multi-Agent Syst.*, vol. 38, no. 1, Dec. 2023, Art. no. 3.

[15] J. Chen, J. Tang, and J. Chen, "Federated meta-learning framework for few-shot fault diagnosis in industrial IoT," in *Proc. IEEE GLOBECOM*, Rio de Janeiro, Brazil, Dec. 2022, pp. 2993–2998.

[16] N. Wang, X. Li, Z. Guan, and S. Yuan, "Fedstream: A federated learning framework on heterogeneous streaming data for next-generation traffic analysis," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 2485–2496, May/Jun. 2024.

[17] Q. Zhang et al., "Maize yield prediction using federated random forest," *Comput. Electron. Agriculture*, vol. 210, May 2023, Art. no. 107930.

[18] J. Zhang, Y. Liu, Y. Hua, and J. Cao, "An upload-efficient scheme for transferring knowledge from a server-side pre-trained generator to clients in heterogeneous federated learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Seattle, WA, USA, Jun. 2024, pp. 12109–12119.

[19] Y. Yang, C. Liu, X. Cai, S. Huang, H. Lu, and Y. Ding, "Unideal: Curriculum knowledge distillation federated learning," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, Seoul, Korea, Apr. 2024, pp. 7145–7149.

[20] X. Liang, J. Tang, and T. Q. S. Quek, "Large-scale decentralized asynchronous federated edge learning with device heterogeneity," in *Proc. IEEE Int. Conf. Commun.*, Denver, CO, USA, Jun. 2024, pp. 1–6.

[21] H. Zhuang et al., "Analytic federated learning," 2024, *arXiv:2405.16240*.

[22] L. Luo and X. Zhang, "Federated split learning via mutual knowledge distillation," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 2729–2741, May/Jun. 2024.

[23] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, "Federated learning with personalization layers," 2019, *arXiv:1912.0081*.

[24] J. Chen, F. Mao, Z. Lv, and J. Tang, "EdgeFD: An edge-friendly drift-aware fault diagnosis system for industrial IoT," in *Proc. IEEE Int. Conf. Commun. Technol.*, Wuxi, China, Oct. 2023, pp. 390–396.

[25] G. Canonaco, A. Bergamasco, A. Mongelluzzo, and M. Roveri, "Adaptive federated learning in presence of concept drift," in *Proc. 2021 Int. Joint Conf. Neural Netw.*, Virtual, Jul. 2021, pp. 1–7.

[26] C. Dupuy et al., "Quantifying catastrophic forgetting in continual federated learning," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, Rhodes Island, Greece, Jun. 2023, pp. 1–5.

[27] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," in *Proc. Nat. Acad. Sci.*, Mar. 2017, vol. 114, no. 13, pp. 3521–3526.

[28] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars, "Memory aware synapses: Learning what (not) to forget," in *Proc. Eur. Conf. Comput. Vis.*, Munich, Germany, Sep. 2018, pp. 139–154.

[29] L. Zhao, Y. Han, A. Hawbani, S. Wan, Z. Guo, and M. Guizani, "Media: An incremental DNN based computation offloading for collaborative cloud-edge computing," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 2, pp. 1986–1998, Mar./Apr. 2024.

[30] A. Mallya and S. Lazebnik, "PackNet: Adding multiple tasks to a single network by iterative pruning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 7765–7773.

[31] H. Zhuang et al., "Online analytic exemplar-free continual learning with large models for imbalanced autonomous driving task," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 1949–1958, Feb. 2025.

[32] D.-W. Zhou, Z.-W. Cai, H.-J. Ye, D.-C. Zhan, and Z. Liu, "Revisiting class-incremental learning with pre-trained models: Generalizability and adaptivity are all you need," *Int. J. Comput. Vis.*, vol. 133, pp. 1012–1032, Mar. 2025.

[33] X. Yang, H. Yu, X. Gao, H. Wang, J. Zhang, and T. Li, "Federated continual learning via knowledge fusion: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 8, pp. 3832–3850, Aug. 2024.

[34] Z. Zhang, Y. Zhang, D. Guo, S. Zhao, and X. Zhu, "Communication-efficient federated continual learning for distributed learning system with non-iid data," *Sci. China Inf. Sci.*, vol. 66, no. 2, Dec. 2022, Art. no. 122102.

[35] Y. Luopan, R. Han, Q. Zhang, C. H. Liu, G. Wang, and L. Y. Chen, "FedKNOW: Federated continual learning with signature task knowledge integration at edge," in *Proc. IEEE Int. Conf. Data Eng.*, Anaheim, CA, Apr. 2023, pp. 341–354.

[36] W. Li, Z. Chen, and G. He, "A novel weighted adversarial transfer network for partial domain fault diagnosis of machinery," *IEEE Trans. Ind. Inf.*, vol. 17, no. 3, pp. 1753–1762, Mar. 2021.

[37] J. Yoon, W. Jeong, G. Lee, E. Yang, and S. J. Hwang, "Federated continual learning with weighted inter-client transfer," in *Proc. Int. Conf. Mach. Learn.*, Virtual, Jul. 2021, pp. 12073–12086.

[38] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778.

[39] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, Virtual, May 2020.

[40] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. Int. Conf. Mach. Learn.*, Sydney, Australia, Aug. 2017, pp. 1126–1135.

[41] M. Riemer et al., "Learning to learn without forgetting by maximizing transfer and minimizing interference," in *Proc. Int. Conf. Learn. Representations*, New Orleans, LA, USA, May 2019.

[42] K. Javed and M. White, "Meta-learning representations for continual learning," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, Vancouver, Canada, Dec. 2019, pp. 1820–1830.

[43] S. Niu, C. Miao, G. Chen, P. Wu, and P. Zhao, "Test-time model adaptation with only forward passes," in *Proc. Int. Conf. Mach. Learn.*, Vienna, Austria, Jul. 2024, pp. 38298–38315.

[44] G. Merlin, V. Lomonaco, A. Cossu, A. Carta, and D. Bacciu, "Practical recommendations for replay-based continual learning methods," in *Proc. Image Anal. Process. Workshops*, Lecce, Italy, May 2022, pp. 548–559.

[45] W. A. Smith and R. B. Randall, "Rolling element bearing diagnostics using the case western reserve university data: A benchmark study," *Mech. Syst. Signal Process.*, vol. 64-65, pp. 100–131, Jun. 2015.

[46] J. Chen, J. Li, R. Huang, K. Yue, Z. Chen, and W. Li, "Federated transfer learning for bearing fault diagnosis with discrepancy-based weighted federated averaging," *IEEE Trans. Instrum. Meas.*, vol. 71, Jun. 2022, Art. no. 3514911.

[47] L. P. Arts and E. L. van den Broek, "The fast continuous wavelet transformation (FCWT) for real-time, high-quality, noise-resistant time–frequency analysis," *Nature Comput. Sci.*, vol. 2, no. 1, pp. 47–58, Jan. 2022.

[48] S. I. Mirzadeh, M. Farajtabar, R. Pascanu, and H. Ghasemzadeh, "Understanding the role of training regimes in continual learning," in *Proc. Neural Inf. Process. Syst.*, Virtual, Dec. 2020, vol. 33, pp. 7308–7320.

[49] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 4510–4520.

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.



Jiao Chen received the B.S. degree in software engineering from Yunnan University, Kunming, China, in 2021. He is currently working toward the Ph.D. degree from the Shien-Ming Wu School of Intelligent Engineering, South China University of Technology, Guangzhou, China. His research interests mainly include industrial edge AI, continual learning, and fine-tuning large language models. He was the recipient of the Best Paper Award at the 2023 IEEE ICCT.



Jiayi He is currently working toward the B.E. degree in intelligent manufacturing engineering with the Shien-Ming Wu School of Intelligent Engineering, South China University of Technology, Guangzhou, China. Her research interests include language models self-correction, long-context large language modeling, and continual learning, focusing on natural language processing and machine learning.



Jianhua Tang (Senior Member, IEEE) received the B.E. degree in communications engineering from Northeastern University, Shenyang, China, in 2010, and the Ph.D. degree in electrical and electronic engineering from Nanyang Technological University, Singapore, in 2015. From 2015 to 2016, he was a Postdoctoral Research Fellow with the Singapore University of Technology and Design, Singapore, and a Research Assistant Professor with the Department of Electrical and Computer Engineering, Seoul National University, South Korea, from 2016 to 2018. He is currently an Associate Professor with the Shien-Ming Wu School of Intelligent Engineering, South China University of Technology, Guangzhou, China. His research interests include wireless communications, edge computing/intelligence, network slicing, and industrial Internet of Things.

He was the recipient of with the 2020 IEEE Communications Society Stephen O. Rice Prize and the 2023 IEEE ICCT Best Paper Award. He is also an Editor of IEEE WIRELESS COMMUNICATIONS LETTERS and *Digital Communications and Networks*.



Weihua Li (Senior Member, IEEE) received the Ph.D. degree in mechanical engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2003. He is currently the Dean and a Professor with the School of Mechanical and Automotive Engineering, South China University of Technology, Guangzhou, China. His research interests include industrial intelligence, industrial Big Data, digital twins, intelligent maintenance & health management, and intelligent connected vehicles.

He is also the Co-Chair of Technical Committee (TC-3) on Condition Monitoring & Fault Diagnosis Instrument, IEEE Instrumentation and Measurement Society (IEEE IM Society). He is a Member of the Editorial Board of IEEE TRANSACTIONS ON INSTRUMENTATION AND MEASUREMENT, IEEE SENSORS JOURNAL, *Chinese Journal of Mechanical Engineering* (CJME), *Journal of Dynamics, Monitoring and Diagnostics* (JDMD), and JOURNAL OF VIBRATION ENGINEERING (JVE, IN CHINESE).



Zihang Yin (Member, IEEE) received the B.E. degree from Chongqing University, Chongqing, China, in 2015, and the master's degree from China Aerospace Science and Technology Corporation (CASC) 9th Academy 771 Research Institute, Xi'an, China, in 2018. He is currently a Researcher with the Research Institute of Industrial Internet of Things, China Academy of Information and Communications Technology, Beijing, China. His research interests include industrial Internet of Things identity carriers, blockchain system design, AI edge computing system design, and solid-state drive (SSD) software algorithm design.

Authorized licensed use limited to: Indian Institute Of Technology Bhilai. Downloaded on September 03,2026 at 10:21:40 UTC from IEEE Xplore. Restrictions apply.